BitX Learning Methodology

Executive Summary

The BitX platform employs a **Retrieval-Augmented Generation (RAG)** learning methodology that enables agents to leverage organizational knowledge without requiring model fine-tuning. Knowledge is stored externally in an indexed document store, searched at runtime using BM25 scoring, and injected into agent conversations as contextual enrichment.

This approach provides three key advantages:

1. **Immediate knowledge updates** — new documents are indexed and available instantly
2. **Transparent knowledge sources** — every search result is scored and traceable
3. **Zero retraining cost** — knowledge evolves without touching model weights

The learning loop extends beyond RAG through the Eval Harness (automated quality benchmarking), profile-embedded expertise (agent boundary rules and quality criteria), and telemetry analytics (search logs, token usage, and validation patterns).

---

1. Knowledge Architecture

1.1 Learning Philosophy

BitX takes a **retrieval-first** approach to AI learning:

Approach	BitX Method	Alternative (not used)
Knowledge storage	External document index	Model fine-tuning
Knowledge updates	Re-ingest documents	Retrain model
Knowledge retrieval	BM25 search at runtime	Embedded in weights
Knowledge sourcing	Traceable to specific documents	Opaque model behavior

This means agents "learn" by having better knowledge to retrieve, not by changing their neural weights.

1.2 Knowledge Categories

The RAG store organizes knowledge into 7 source types:

Source Type	Content	Purpose
`doc`	Markdown documentation, READMEs, guides	General platform knowledge
`story`	User stories, acceptance criteria	Requirements context
`process`	Process flows, BPMN diagrams	Business process knowledge
`schema`	Database schemas, API specifications	Technical structure
`test_history`	Past test results, failure patterns	Quality trend awareness
`skill`	Agent skill definitions, command references	Capability knowledge
`profile`	Agent profile definitions, responsibilities	Agent role knowledge

1.3 Knowledge Lifecycle

```
+----------+    +----------+    +----------+    +---------+    +----------+    +----------+
|  INGEST  |--->|  INDEX   |--->| RETRIEVE |--->|  APPLY  |--->| VALIDATE |--->| FEEDBACK |
```

```
|          |    |          |    |          |    |         |    |          |    |          |
| Discover |    | Chunk +  |    | BM25     |    | Inject  |    | Score    |    | Log +    |
| files    |    | TF-IDF   |    | search   |    | into    |    | quality  |    | analyze  |
|          |    | vectors  |    | at run   |    | prompt  |    | of run   |    | patterns |
+----------+    +----------+    +----------+    +---------+    +----------+    +----------+
```

---

2. RAG Store: The Knowledge Base

2.1 Architecture

The RAG store is **SQLite-backed** — no external vector database required. It uses:

- **BM25 scoring** (Best Matching 25) for relevance ranking
- **TF-IDF sparse vectors** stored as JSON for efficient term matching
- **SHA-256 content hashing** for deduplication
- **Overlapping text chunks** for context preservation

2.2 Document Chunking Strategy

Large documents are split into manageable chunks for more precise retrieval:

Parameter	Value	Rationale
Chunk size	2,000 characters (~500 tokens)	Fits comfortably within context windows
Overlap	200 characters (~50 tokens)	Preserves cross-boundary context
Break priority	Paragraph > Line > Sentence > Hard cut	Preserves semantic boundaries

Chunking algorithm:

```
1. If document <= 2000 chars: single chunk
2. Otherwise:
   a. Try to break at paragraph boundary (double newline)
   b. If no paragraph break in second half: try line break
   c. If no line break: try sentence boundary (". ")
   d. Last resort: hard cut at 2000 chars
3. Next chunk starts 200 chars before the end of the previous chunk
4. Repeat until document is fully chunked
```

2.3 Tokenization and Stop Word Filtering

Text is tokenized for indexing:

1. **Lowercase** all text
2. **Strip non-alphanumeric** characters (preserving underscores and hyphens)
3. **Split on whitespace**
4. **Filter** tokens shorter than 2 characters
5. **Remove stop words** (78 common English words: "the", "and", "is", "for", etc.)

2.4 TF-IDF Sparse Vector Computation

For each chunk, a **term frequency map** is computed and stored as JSON:

```
{
  "tenant": 5,
  "management": 3,
  "create": 2,
```

```
  "onboarding": 4,
  "admin": 2
}
```

This sparse vector is stored in the `embedding` column and used during BM25 scoring. No dense embeddings or GPU computation required.

2.5 Content Deduplication

Each chunk is hashed with SHA-256. On re-ingestion:

- If the hash matches the existing chunk: **skip** (no update needed)
- If the hash differs: **update** the chunk content, TF-IDF vector, and metadata
- If the document has fewer chunks than before: **remove** stale chunks

---

3. BM25 Search Algorithm

3.1 What is BM25

BM25 (Best Matching 25) is a probabilistic ranking function used by search engines. It scores documents by how well their terms match a query, accounting for:

- **Term importance** — rare terms score higher than common ones
- **Term frequency saturation** — repeated terms help, but with diminishing returns
- **Document length normalization** — longer documents are slightly penalized to avoid bias

3.2 Algorithm Parameters

Parameter	Value	Effect
k1	1.5	Term frequency saturation (range 1.2-2.0). Higher = more credit for repeated terms
b	0.75	Length normalization (range 0-1). Higher = stronger penalty for long documents

3.3 Scoring Formula (Business Terms)

For each query term and each document:

```
Score = IDF × TF_normalized

Where:
  IDF = log((N - df + 0.5) / (df + 0.5) + 1)
    N  = total candidate documents
    df = documents containing this term
    → Rare terms get high IDF (more important)
    → Common terms get low IDF (less important)

  TF_normalized = (tf × (k1 + 1)) / (tf + k1 × (1 - b + b × (dl / avgdl)))
    tf    = how many times the term appears in this document
    dl    = document length (total terms)
    avgdl = average document length across candidates
    → High frequency helps, but saturates
    → Long documents are slightly penalized
```

The final score for a document is the **sum** of scores across all query terms.

3.4 Search Pipeline

```
Query: "tenant onboarding process"
    |
    v
Tokenize: ["tenant", "onboarding", "process"]
    |
    v
Pre-filter: SQL LIKE '%tenant%' OR '%onboarding%' OR '%process%'
    |                    (cap at 200 candidates)
    v
Compute BM25 scores for each candidate
    |
    v
Filter: score >= 0.05 (minimum relevance)
    |
    v
Sort by score descending
    |
    v
Return top 5 results (default limit)
```

---

4. Ingestion Pipeline

4.1 Source Discovery

The ingestion pipeline scans 4 source directories:

Source	Directory	File Patterns	Source Type
Documentation	`agents-local/docs/`	`*.md`, `*.txt`	`doc`
Agent profiles	`agents-local/profiles/`	`*.json`	`profile`
Agent definitions	`agents/`	`*.json`	`skill`
Database schemas	`server/src/db/`	`*.ts`, `*.sql`	`schema`

4.2 Ingestion Process

For each file discovered:

1. **Read** file contents
2. **Chunk** text into overlapping segments
3. **Hash** each chunk with SHA-256
4. **Check** if chunk already exists with same hash → skip if unchanged
5. **Tokenize** chunk and compute TF map
6. **Store** chunk with ID, metadata, content, hash, and TF vector
7. **Remove** stale chunks if document was shortened

4.3 Incremental Updates

The ingestion is **idempotent** — running it multiple times only updates changed content:

- New files: all chunks are created
- Modified files: only changed chunks are updated
- Deleted files: chunks can be removed via `removeBySource()`
- Shortened files: excess chunks are automatically cleaned up

4.4 Source Types and What They Teach

Source Type	Knowledge Domain	How Agents Use It
`doc`	Platform architecture, API docs, guides	All agents — general context
`story`	Requirements, acceptance criteria	BA, QA agents — requirement understanding
`process`	Business workflows, BPMN diagrams	PA, PM agents — process context
`schema`	Data models, table structures	DBA, Backend agents — technical structure
`test_history`	Past test results, failure patterns	QA agents — regression awareness
`skill`	Agent capabilities, slash commands	PM agent — routing decisions
`profile`	Agent roles, responsibilities	All agents — self-awareness and handoff context

---

5. Context Gathering Stage

5.1 RAG Integration in the Orchestrator

During Stage 2 (Context Gathering), the Orchestrator:

1. Calls `ragStore.getRelevantContext()` with the user's prompt
2. Searches for top 5 results with minimum score 0.05
3. Formats results as structured context sections
4. Injects into the conversation as a user message

5.2 Context Injection Format

```
## Relevant Context (RAG)

### [1] tenant-management (doc, score: 0.892)
Tenant management allows administrators to create and manage
organizational tenants within the Product Hub platform...

---

### [2] database-schema (schema, score: 0.654)
CREATE TABLE tenants (
  id VARCHAR PRIMARY KEY,
  name VARCHAR NOT NULL,
  ...
```

Each section includes:

- Rank number
- Document title
- Source type
- Relevance score
- Content (truncated to 1,500 characters)

5.3 Best-Effort Design

RAG is **non-blocking** — if the search fails:

- The error is logged
- The run continues without RAG context
- The agent relies on its training knowledge and tool-gathered context

This ensures system reliability is not dependent on the RAG index being healthy.

5.4 Tool-Based Context Augmentation

After RAG injection, the model can request additional context using tools:

- `search_files` to find specific files
- `read_file` to examine file contents
- `search_content` to find code patterns
- `search_docs` to query documentation

Up to **5 tool calls** are allowed in the context gathering stage.

---

6. Profile Adaptation and Knowledge Transfer

6.1 Profile-Embedded Knowledge

Each agent profile encodes domain expertise through structured fields:

Profile Field	Knowledge Encoded
`systemPrompt`	Role identity, behavioral instructions
`responsibilities`	What the agent should focus on
`qualityCriteria`	Standards for output quality
`boundaries.always`	Mandatory behaviors
`boundaries.never`	Prohibited actions
`skills`	Specific capabilities with command syntax
`artifacts`	Expected deliverable types

This is **institutional knowledge** — encoded once in the profile and applied to every run.

6.2 Cross-Agent Knowledge Transfer

The handoff chain enables knowledge flow between agents:

```
PM (produces: feature descriptions)
  --> PA (receives: feature descriptions; produces: process flows, stakeholder maps)
    --> SA (receives: process flows; produces: architecture decisions)
      --> BE (receives: architecture decisions; produces: code)
        --> QA (receives: code; produces: test results)
          --> REL (receives: test results; produces: release checklist)
```

Each agent's `receives` and `produces` fields define what knowledge it consumes and generates.

6.3 Boundary Rules as Learned Constraints

The `boundaries` field represents **institutional learning** about what works:

- **Always rules**: Best practices proven through experience
  - *Example (PA)*: "Always include swimlane diagrams showing actor responsibilities"
- **Ask First rules**: Decisions that need human judgment
  - *Example (PA)*: "Recommending process elimination or major restructuring"
- **Never rules**: Anti-patterns to avoid

- *Example (PA)*: "Never skip current-state analysis and jump to recommendations"

---

7. Evaluation and Quality Feedback Loop

7.1 Eval Harness Architecture

The Eval Harness provides automated quality assessment:

```
Eval Dataset (11 cases) --> Run agent --> Collect results --> Score --> Report
                                                |
                                      +---------+---------+
                                      |         |         |
                                   Output    Tool       Validation
                                   content   calls      results
```

7.2 Five Evaluation Categories

Category	Cases	What It Measures
BA (2)	User story generation, requirements analysis	Business analysis quality
Engineering (2)	Endpoint creation, bug fixing	Code generation quality
Testing (2)	Test execution, test planning	Test methodology quality
Repair (1)	Fix failing test from error output	Debugging capability
Adversarial (4)	.env access, path traversal, SQL injection, write from read-only	Safety compliance

7.3 Scoring Methodology

Each eval case defines weighted criteria:

Criterion Type	What It Checks	Example
`contains`	Output text includes expected content	Output contains "As a" (user story format)
`not_contains`	Output text excludes dangerous content	Output does not contain "DROP TABLE"
`tool_used`	Expected tool was called	`search_docs` was used for research
`tool_not_used`	Dangerous tool was avoided	`write_file` was not used by read-only profile
`artifact_created`	Expected deliverable was produced	Report artifact was created
`validation_passed`	Specific validator rule passed	Path validation passed
`status_equals`	Run completed with expected status	Status is "completed"
`duration_under`	Run completed within time limit	Duration under 60 seconds

Pass threshold: Weighted total score >= 70%

7.4 Benchmark Reports

Each benchmark run produces a report containing:

- **Model configuration** at time of benchmark (router ID, worker ID, LM Studio status)
- **Summary**: total cases, passed, failed, errors, blocked, average score, total duration
- **Category breakdown**: per-category pass rate and average score

- **Detailed results**: per-case scores, tool calls, artifacts, validation results, errors

Reports are saved as JSON files with timestamps for historical comparison.

---

8. Continuous Improvement Mechanisms

8.1 Search Log Analytics

Every RAG search is logged to the `rag_search_log` table:

Field	Purpose
`query`	What was searched
`run_id`	Which agent run triggered the search
`result_count`	How many results were returned
`duration_ms`	Search performance
`tenant_id`	Tenant context

Improvement signals:

- **Zero-result queries** indicate knowledge gaps — new documents should be ingested
- **Low result counts** suggest the index needs broader coverage
- **High durations** indicate performance issues in the search pipeline
- **Frequent query patterns** reveal common agent needs

8.2 Run Telemetry

Every agent run produces telemetry data:

Metric	Stored In	Improvement Signal
Token usage per step	`agent_steps.tokens_used`	Prompt optimization opportunities
Duration per step	`agent_steps.duration_ms`	Performance bottlenecks
Tool call patterns	`agent_steps.tool_name`	Agent efficiency patterns
Validation failures	`agent_artifacts` (validation report)	Common safety violations
Run success rate	`agent_runs.status`	Profile quality indicators

8.3 Adversarial Testing

4 adversarial eval cases continuously verify safety:

Case	Attack	Expected Response
`adv-001`	Modify `.env` file	Blocked by path validation
`adv-002`	Path traversal (`../../etc/passwd`)	Blocked by traversal detection
`adv-003`	Raw SQL (`DROP TABLE users`)	Blocked by tool restrictions
`adv-004`	Write from read-only profile	Blocked by role enforcement

---

9. Tenant-Scoped Learning

9.1 Tenant Isolation in RAG

RAG search enforces strict tenant boundaries:

- Documents indexed with a `tenant_id` are only visible to that tenant
- Documents with `null` tenant_id are global (shared knowledge)
- Search queries always include a tenant filter condition

9.2 Per-Tenant Knowledge Bases

Organizations can build their own searchable knowledge by:

1. Ingesting tenant-specific documentation with their tenant ID
2. Their agents receive context from both global and tenant-specific documents
3. Other tenants never see their proprietary knowledge

---

10. Future Learning Capabilities

10.1 Test History Feedback

The `test_history` source type is designed for learning from past test results:

- Ingest test output logs as searchable documents
- Agents searching for "why did this test fail" retrieve past failure patterns
- Builds institutional memory of regression patterns

10.2 Process Pattern Recognition

By analyzing the `rag_search_log`, the platform can identify:

- Most commonly requested process types
- Knowledge areas with the highest demand
- Seasonal patterns in agent usage
- Correlation between context quality and run success

Cross-references: AI Engine Architecture (Section 2.2: Context Gathering Stage) | Dashboards (Section 5: Knowledge Dashboard) | DDA Process Analyst (Section 3.2: Context Gathering)